The Journal of Open Source Software

pyCoastal: a Python package for numerical modeling in coastal engineering

Stefano Biondi [1]

1 University of Florida, United States

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 19 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

pyCoastal is an open-source Python package for the generation of numerical simulations of coastal hydrodynamics and scalar transport processes. The package provides a modular and extensible platform for solving linear and nonlinear partial differential equations (PDE) relevant to coastal processes, including wave propagation, pollutant dispersion, and viscous fluid dynamics. The framework involves grid generation, boundary condition management, numerical operators, and the selection of time integration schemes. Physical modules, such as literature formulations for common coastal processes, are implemented as standalone classes and can be easily composed or extended for prototyping and research. pyCoastal emphasizes clarity and reproducibility, with a strong focus on clean code structure and pedagogical transparency. It is particularly suited for simple applications in real case scenarios and as educational tool in coastal engineering, fluid mechanics, and numerical modeling.

Statement of need

Numerical modeling of coastal processes, such as wave propagation, shallow water dynamics, and pollutant transport, typically relies on specialized software frameworks that are often complex to configure, extend, or adapt to new applications. Tools like SWAN, ADCIRC, and OpenFOAM, although powerful and detailed, present significant barriers due to their steep learning curves and rigid internal structures. More computationally demanding models do not necessarily yield higher accuracy, especially when simpler models are tuned effectively (Lashley et al., 2020). In fact, some processes can be modeled simply and still produce accurate, useful results when the simpler model is well tuned. Furthermore, successful calibration of numerical models hinges on the ability to accurately represent key physical processes and structural features, which can be hard to achieve in large modeling frameworks (Simmons et al., 2017). This complexity poses challenges both for young research and industry applications aiming to prototype models rapidly, as well as for instructors seeking clear, demonstrable tools for teaching coastal processes.

To solve this issue, pyCoastal offers a lightweight and modular coastal modeling framework fully in Python. It is designed to prioritize clarity and reproducibility, allowing users to define simulations through simple YAML configuration files and execute them with minimal setup. The codebase provides reusable components for grid generation, numerical operators, time integration schemes, and boundary condition handling, supporting both classical and custom physical models with ease. Its structure is designed to support both research and industry applications or instructional use in topics such as coastal hydrodynamics, numerical modeling, and environmental fluid mechanics. Moreover, pyCoastal integrates numerous established coastal-engineering formulations, such as wave run-up, sediment transport, and boundary layer calculations, enabling users to compute essential coastal parameters with ease.

As a result, pyCoastal functions as a versatile library suitable for both academic research and

industrial applications in coastal engineering.

How does it work

In this section, a simple example application of the `pyCoastal` framework is presented.

Irregular wave propagation on a 2D domain

This example shows how pyCoastal can create and propagate a realistic wave signal based on an oceanographic spectrum in a 2-dimensional domain. The example builds a time series of surface elevation by sampling a chosen spectrum (built-in as internal function in `pyCoastal`), assigning random phases, and summing the individual harmonic components. The user specifies inputs such as significant wave height, peak period, duration, and time step. The routine then returns the time vector and the corresponding free-surface signal that can be used as boundary forcing in numerical simulations (Figure 1). This example demonstrates how a spectral description of sea states can be converted into a synthetic wave time series suitable for coastal modeling. Additionally, it is given to the user the ability to add monitoring point (buoy-style) everywhere in the domain.

The inputs for the domain and wave generation are taken from a YAML file. These YAML files share the names of the corresponding examples and are located in `/examples/config/`. The YAML reads:

```yaml
grid:
  nx: 200
  ny: 200
  dx: 1.0
  dy: 1.0

physics:
  gravity: 9.81
  depth: 5.0

forcing:
  type: jonswap   # options: pm or jonswap
  gamma: 3.3
  Hs: 0.5         # significant wave height (m)
  Tp: 3.0         # peak period (s)

solver:
  dt: 0.1
  duration: 60.0

output:
  gauge: [100, 100]  # buoy-style observation point
```

JONSWAP spectrum

As a reference, it is explained the formulation used in the source code to calculate the Jonswap spectrum (Hasselmann et al., 1973). This spectrum modifies the Pierson-Moskovitz spectrum with a peaked enhancement factor as:

$$S_J(f) = S_{PM}(f)\,\gamma^{\exp\left[-\frac{1}{2\sigma^2}\left(\frac{f}{f_p}-1\right)^2\right]},$$

where: - $S_J(f)$ is the JONSWAP spectral energy density [m²/Hz] - -f is the frequency - γ is the peak enhancement factor (typically 3.3, but can be customized to change the spectrum shape)
- σ is the spectral width parameter, with $\sigma = 0.07$ for $f < f_p$ and $\sigma = 0.09$ for $f > f_p$

Once $S(f)$ is defined, the surface elevation time series is:

$$\eta(t) = \sum_i A_i \cos(2\pi f_i + \phi_i),$$

where: - $\eta(t)$ is the free-surface elevation at time t [m]
- A_i is the wave amplitude for frequency f_i [m]
- ϕ_i is a random phase shift in $[0, 2\pi]$
- Δf is the frequency resolution

The amplitudes A_i are given by:

$$A_i = \sqrt{2\, S(f_i)\, \Delta f}.$$

Irregular wave field output of the example

Figure 1: Irregular wave field output of the example

Figure 1: Irregular wave field produced in the example.
The left panel shows a top view of the wave field with waves entering from the southern boundary. Side boundaries use free slip, and the northern boundary absorbs outgoing waves. The right panel shows the surface elevation over time at selected observation points.

To test it, run this example:

```
python examples/wave2D_irregular.py
```

This example provides a simple application of `pyCoastal`. However, the ability to manipulate spectra, generate time series and visualize wave propagation is fundamental in coastal engineering, and hands-on experimentation is essential for learning. To make the tool more representative of real situations, users can generate or import custom bathymetry and apply it to the computational domain.

Conclusion Future directions

In this work, the Python module for coastal processes `pyCoastal` was presented. The module is a simple and modular Python framework for building numerical models of coastal hydrodynamics and transport. Its design emphasizes clarity, flexibility and ease of use, allowing users to prototype simulations, explore physical processes and support instructional needs with minimal overhead. Future development will include wave energy dissipation and breaking models, more complete nearshore hydrodynamics, and sediment transport processes. Additional coastal-engineering formulations and analysis tools will also be integrated, further extending the range of research and educational applications that pyCoastal can support.

Acknowledgements

I thank professors Alberto Canestrelli (UF) and Donald Slinn (UF) for the teachings in the numerical modeling and Hydrodynamics field.

References

see paper.bib

Hasselmann, K., Barnett, T., Bouws, E., Carlson, H., Cartwright, D., Enke, K., Ewing, J., Gienapp, H., Hasselmann, D., Kruseman, P., Meerburg, A., Müller, P., Olbers, D., Richter, K., Sell, W., & Walden, H. (1973). Measurements of wind-wave growth and swell decay during the joint north sea wave project (JONSWAP). *Deutsche Hydrographische Zeitschrift*, *8*, 1–95.

Lashley, C. H., Zanuttigh, B., Bricker, J. D., Meer, J. van der, Altomare, C., Suzuki, T., Roeber, V., & Oosterlo, P. (2020). Benchmarking of numerical models for wave overtopping at dikes with shallow mildly sloping foreshores: Accuracy versus speed. *Environmental Modelling & Software*, *130*, 104740. https://doi.org/10.1016/j.envsoft.2020.104740

Simmons, J. A., Harley, M. D., Marshall, L. A., Turner, I. L., Splinter, K. D., & Cox, R. J. (2017). Calibrating and assessing uncertainty in coastal numerical models. *Coastal Engineering*, *125*, 28–41. https://doi.org/10.1016/j.coastaleng.2017.04.005

DRAFT